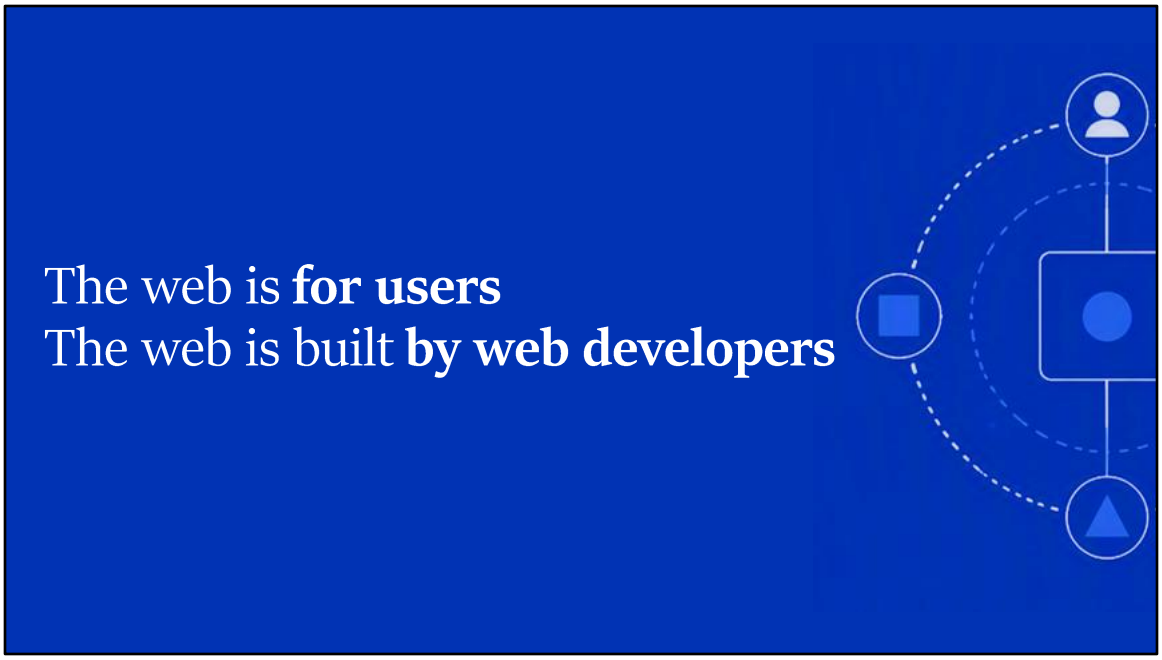


Stop scraping the web - Start calling it instead

Let’s talk about agentic web browsing

Today, I want to talk to you about agentic web browsing.



Before I do, let's talk about the web first.

The web is for users.
The web is built by web developers.

For users:
It's where everything happens nowadays. It's what most people use on their devices. It's how most apps, desktop or mobile, are built.
Picture yourself coming to work in the morning and opening your laptop: the browser is probably the first thing you open, and often, the only thing you'll use throughout the day. And even if it's not, the other apps you use are likely built on web technologies.

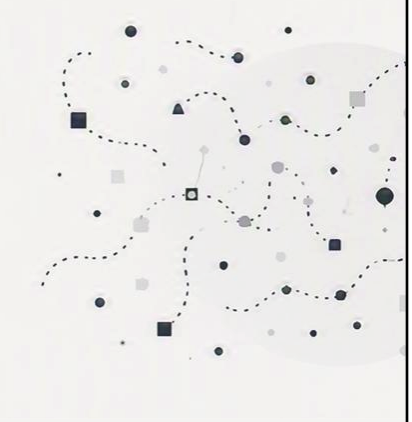
By developers:
the web is also built by web developers. It seems maybe obvious.
They're build the sites, apps, and experiences that users interact with.
They define how users accomplish those tasks, what actions are available, and how to navigate through the content, etc.

Agentic web browsing, meaning when agents navigate websites on your behalf, should be no different.
It should be FOR users, and BY developers.

The web is a fantastic resource of billions of interconnected documents.
It needs to part of the agentic future.

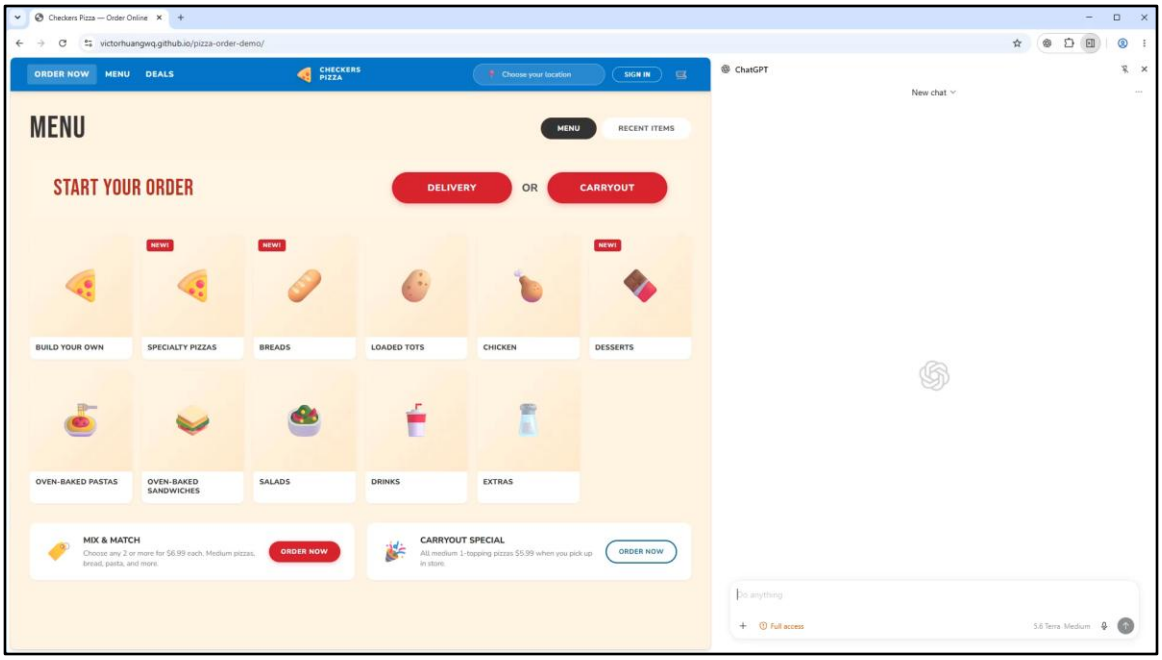
It already is, but we can do a lot better.

Agentic browsing works **despite** websites, not **with** them



Today's agentic browsing works FOR users, so we have that first part
but not with websites, so not helped by web developers.
It works despite websites, not with them.

Browser agents, today:
scrape the DOM
inspect accessibility trees
analyze screenshots
simulate clicks, scroll around, simular keyboard events in input fields, and guess user intent and how sites are navigated.



[Voice over the steps.
Show that the extension drives the mouse and keyboard.
Figures out the UI.
Makes a mistake in the menu.]

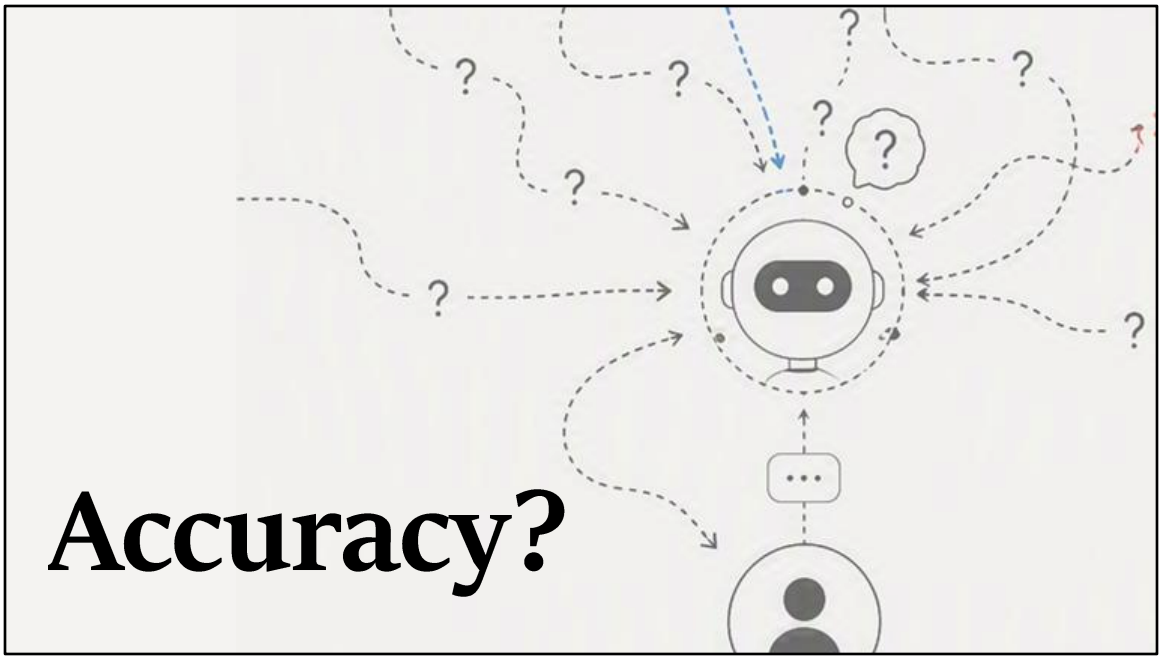


This is very impressive.

But, fundamentally, this is an adaptation layer on top of websites, a patch.
It needs to exist because the website has not yet been given a native way to participate in this whole dance.

The tools that agentic browsing is based on aren't capable enough,
forcing the agent to act exactly as a human would,
instead of interacting through APIs with purpose, that were built by developers.

Frankly, it doesn't seem very optimal for an agent to act on behalf of the human by doing things exactly as the human would have,
but slower,
and consuming a lot of tokens too.



Cause cost and speed aren’t the only potential problems.

Accuracy is a problem too.

The agent is relying on the DOM.
Which we know is often just a bunch of DIVs where developers should have used more semantic elements instead.

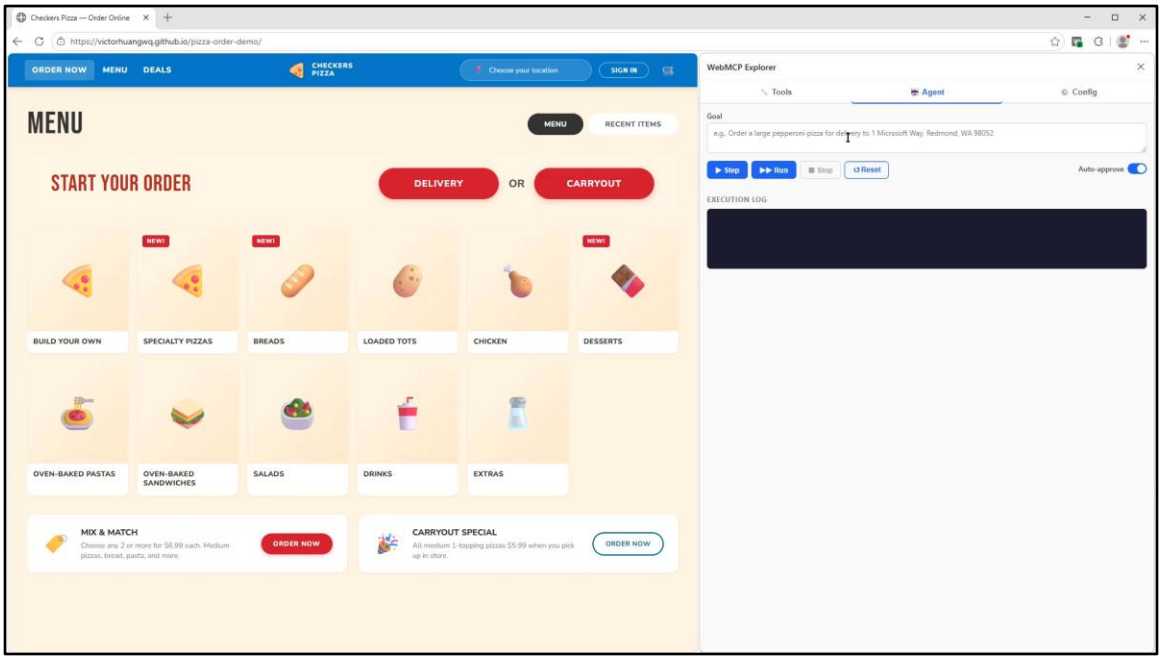
The agent also uses the AT.
And we know it’s very incomplete, because developers don't always take the time to use the right elements and attributes.

And screenshots can be deceiving to agents.

The whole thing comes with brittleness.



And that's because developers are largely excluded from the loop today. What if websites stopped being passive surfaces and became active participants?



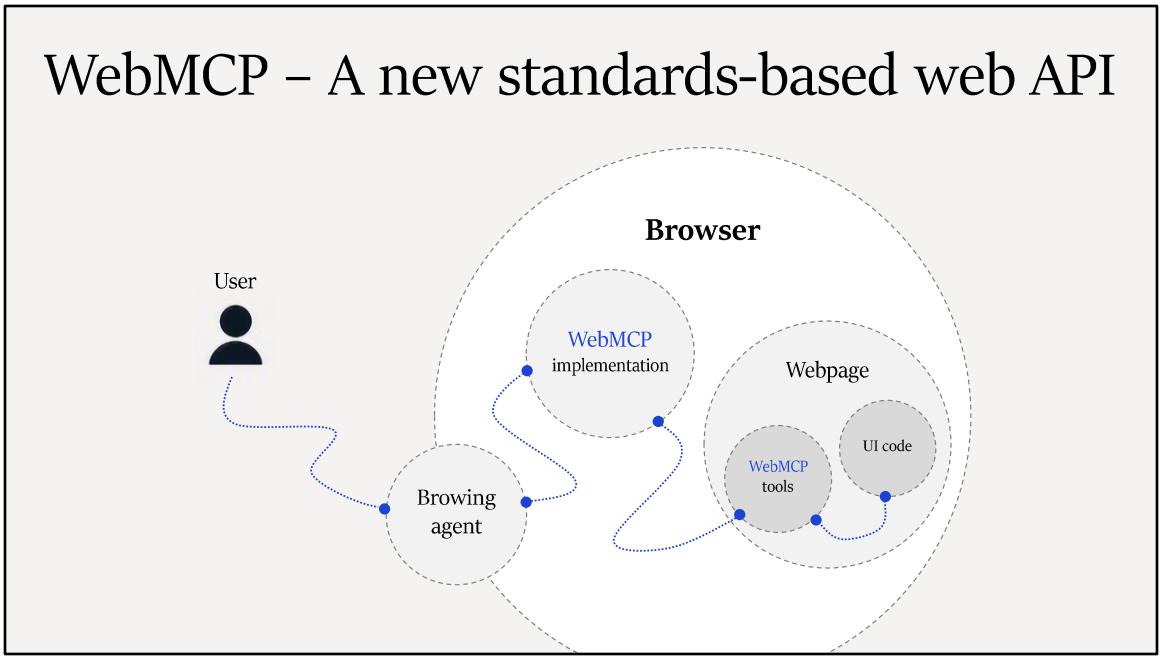
Where WebMCP comes in
Demo
That's where WebMCP comes in.

Let's start with a demo.

[speak over the steps]

Very different from the previous demo, right?
Much faster and token efficient, and a lot more stable too.
There's no brittleness due to having to figure out the UI.

About token consumption, we did some tests internally to compare.
Obviously, this will depend on the scenario, and the exact final implementation, but in our tests, and you'll have to take my word for it, the second demo, using WebMCP, consumed half as many tokens than the first one.



How does this work?
I'll go quick on this, this is a short talk, but I'll give you links to find out more at the end.

WebMCP is an upcoming standards-based Web API that lets you, web developers, expose the capabilities of your site to in-browser agents.

You can almost think of this as a MCP server for a webpage, which the browser starts for you, exposing tools which the webpage registered, to an MCP client. Except that the comparison to MCP ends here. Even though the WebMCP name includes MCP in it, the 2 technologies are not the same, and they don't compete with each other. They complement each other. They solve different problems.

WebMCP was heavily inspired by MCP. But there's no real client and server involved here in the MCP sense.

The "client", so to speak, here, is not whatever you want, such as the Claude app for example. Instead it's whatever browsing agent the browser is running. The demo extension I showed before is one example. But over time, this could be

Copilot in Edge for example.
And the "server", in MCP terms, is the browser itself, exposing tools from the currently visible page.

WebMCP – How to?

Option 1 – Declarative HTML API

```
<form toolname="…" tooldescription="…">
  <input type="text" name="q" toolparamdescription="…">
  …
</form>
```

As a web developer, you expose tools for your site in 2 ways:

1. Declaratively, by adding attributes to your HTML form elements.

Very simple, straightforward, and easy to add to your existing code, without changing anything.
Suddenly, your existing forms become easily usable by the agent running in the browser, but also remain usable by your real users.
It's a way to augment the capabilities of your site for agents, without changing the experience for your users.

WebMCP – How to?

Option 2 – Imperative JS API

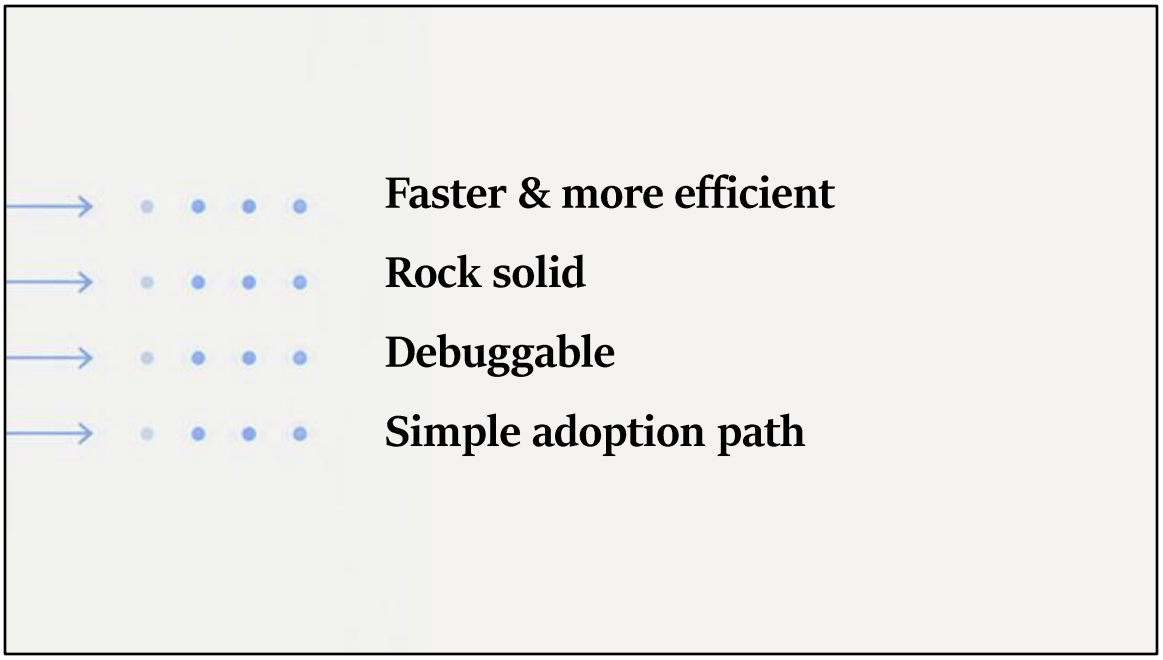
```
document.modelContext.registerTool({
  name: "…",
  description: "…",
  inputSchema: { … },
  execute() { /* Run UI code here */ }
});
```

2. Imperatively, by calling a JavaScript API: document.modelContext.registerTool.

This is a more flexible approach, allowing you to define tools that aren't necessarily tied only to a form.
If you have a lot of client-side JS logic in your web app, this allows you to reuse it as is.

In both cases, you give each tool a name and description, optional parameters, and a JSON schema for the input and output of the tool, very similar to MCP.

And then, all this needs is a browser agent that is aware of WebMCP. When that's the case, the agent can ask the browser for tools on the page, and if any, decide to use them to complete a user task.



What are the benefits of this approach?

- Obviously, we saw in the demo that it can lead to much faster and more efficient UX.
  Because WebMCP tools don't have to follow UI steps 1 to 1 as we saw with the previous demo.
  You can make your WebMCP tool call whatever code on the client-side you want, possibly chaining them to do multiple things at once.
  In the pizza ordering demo, the agent was able to add both my delivery address and select the right pizza in one go.

- Rock solid
  No flaky scenarios involved.
  Tools do exactly what they are supposed to. They're code.
  Obviously you can have bugs in the code, but those you can fix.

- More debuggable too
  You can simply inspect your page to discover tools, schemas, review runtime errors and logs, etc.

and reason about failures instead of replaying flaky click scripts.
But also, the Application tool in DevTools now has a WebMCP section, that lets you inspect and run tools!
Also, the Lighthouse tool now gives you an agentic browsing score, that's based on WebMCP.

- Adoption path
You can expose existing app logic as WebMCP tools incrementally without breaking your current UX.
In fact, this is made to work with the existing UX, not replace it.



But, why not just MCP then?

Fair question.
Sites tend to rely on backend APIs anyway, why not just expose those to agents via an MCP server?

- WebMCP is not designed to compete with this or replace it.
  WebMCP occupies a different place.

- It's optimized for human-in-the-loop workflows,
  reusing existing UI and the shared client-side logged-in state.
  Human does a thing, then asks the agent to take over for another part, then takes control again, etc.
  Imagine a photo editing web app where the user does some edit on one photo, and then asks the agent to do the same on a batch of photos.
  So it's really about collaboration on a single state, where both user and agent work together as one.

- UI is the product.

For many sites and apps, the UI isn't just a thin state-less client over an server-side API.

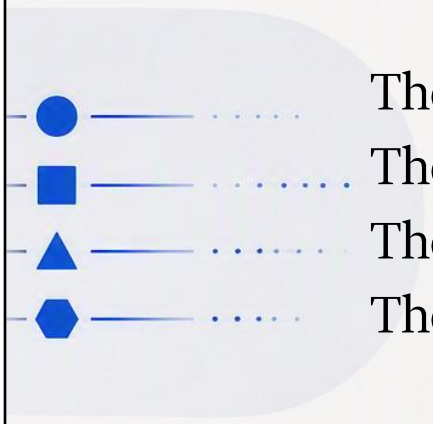
You often can't do everything via the server API.

And, not all apps have a server API.

Often, the client has the logic, and the server just stores some end state.

WebMCP allows you to reuse all of this.

Where WebMCP shines

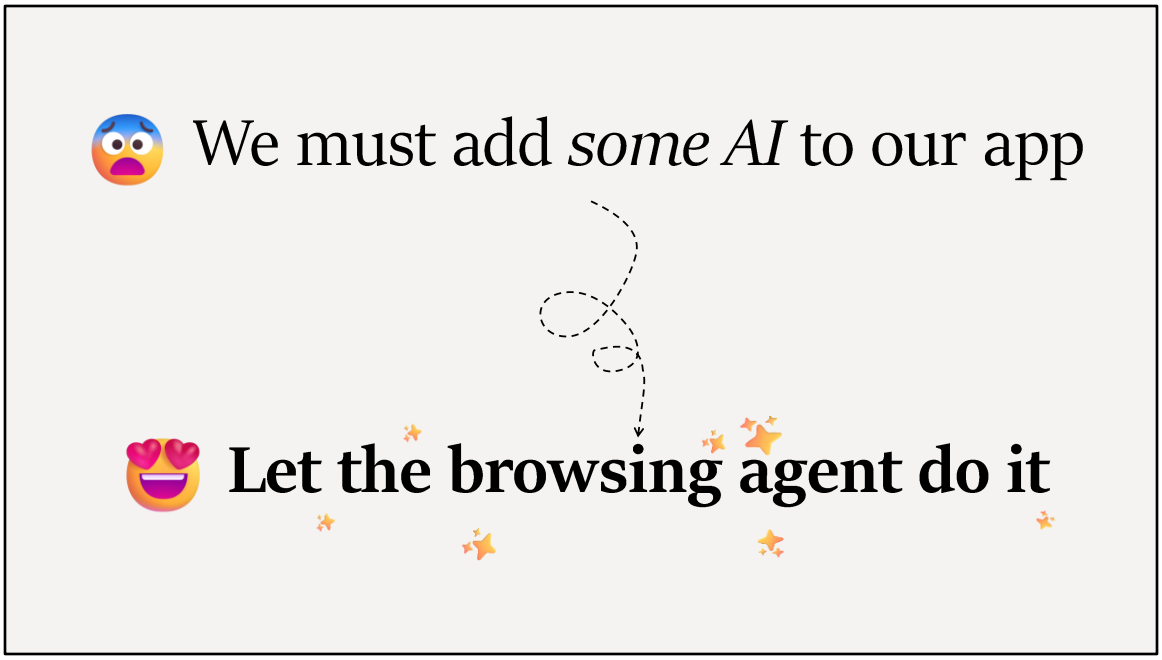


The app is heavily client-side
The UI itself matters
The state lives in the browser
The user and the agent collaborate

WebMCP is really useful when:

- the application is heavily client-side,
- the UI itself matters,
- the state mostly lives in the browser,
- the user and the agent are collaborating.

But also, when you don’t want to pay the network cost of supporting an MCP endpoint.



But, it's also a really pragmatic way to get start with getting your app agent-ready.

In fact, over the past few years, I've heard many folks want to "add some AI to their apps"
Being asked by their bosses.
And end up with a chat bot sidebar.
Which is not what users want.

WebMCP changes the equation.
You can keep on focusing on what you already do best: your app's code.
Expose it as WebMCP tools
and then let browser agent do what _*it*_ do best.

Bonus: it doesn't require you to become AI experts.



So how real is this?

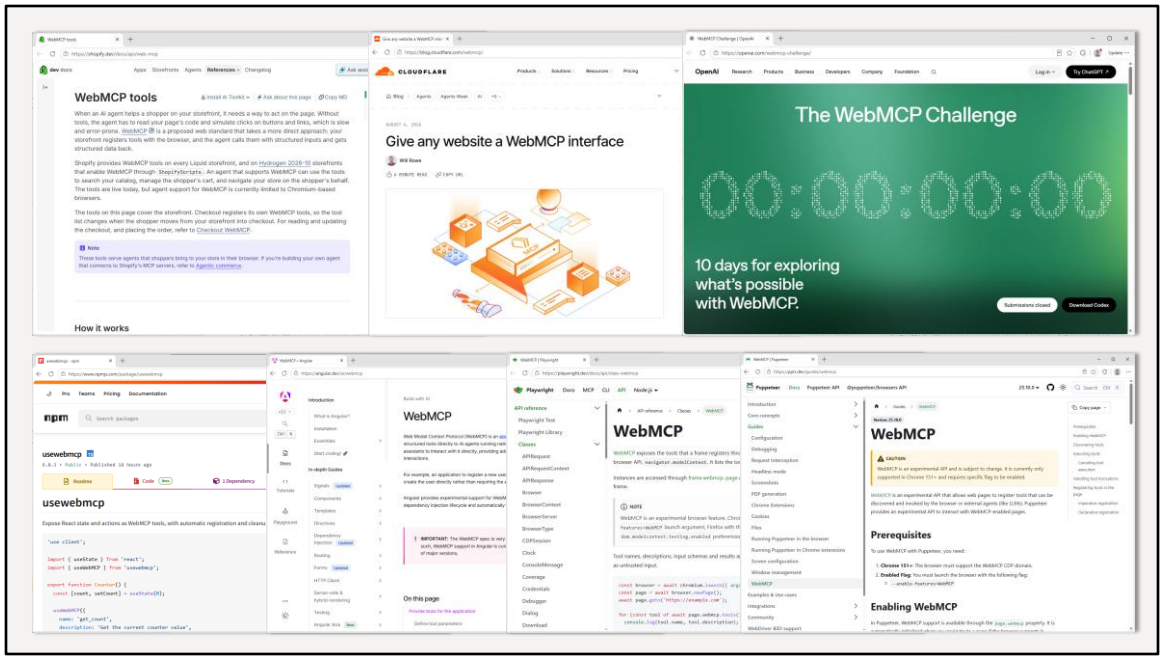
The specification is in progress, but already pretty advanced.
It's being designed inside the W3C WebML Community Group.

Major contributors are my team at Microsoft, and Google, but also community members,
Other browsers are involved too.

BUT, this doesn't mean you can't already use the feature.
On the web, the best way to make progress is to ship early and often,
so developers can try the feature out and give feedback,
which can inform the spec and help it evolve.

WebMCP is implemented in Chromium-based browsers, so Edge and Chrome, behind a flag for now.

But also, WebMCP is now seeing a very rapid ecosystem adoption.
You might have seen a few announcements about it in recent weeks [next slide]



Shopify added support for WebMCP on all their storefronts, to search for products, add them to cart, etc.

Cloudflare added something which automatically creates WebMCP tools for your site.

OpenAI added support for it in ChatGPT and Codex.
And ran a hackathon “WebMCP challenge”, which more than 7000 people participated in.

Popular JS frameworks like React and Angular have built-in support for WebMCP tools.

Front-end testing libraries like Playwright and Puppeteer also now let you test WebMCP code.

And more …

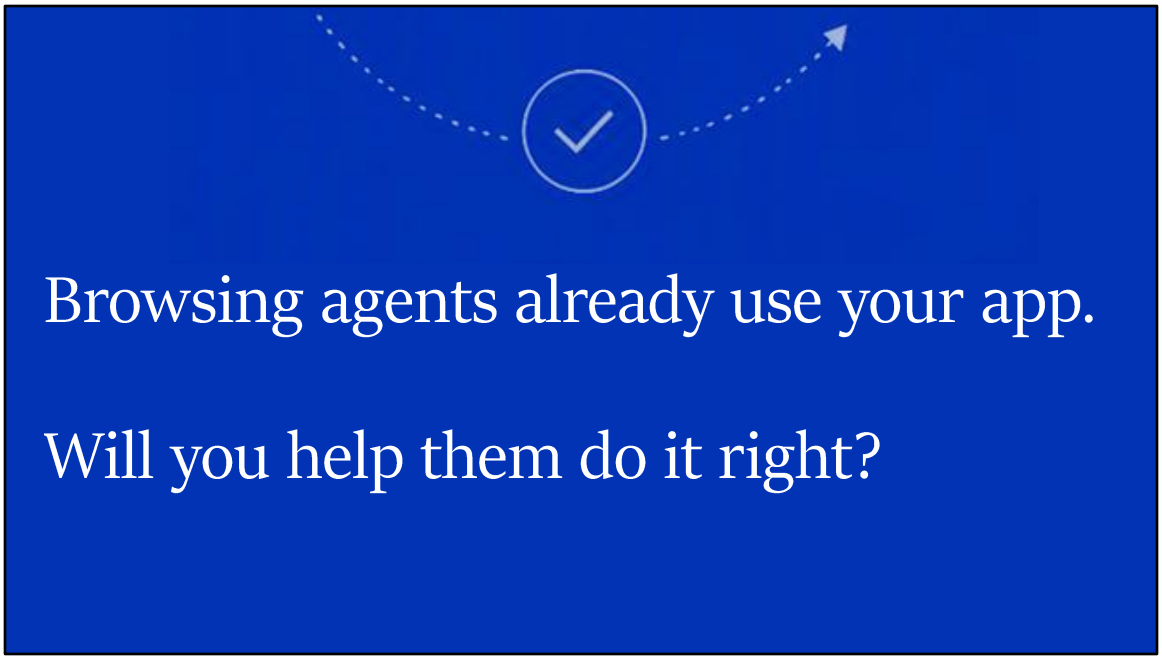
Try WebMCP today!

1. Open `about://flags`
2. Enable `WebMCP for testing`
3. Install WebMCP explorer
4. Test the Pizza site

Go to about://flags
Enable …

The demo I showed earlier is real, I recorded it in Microsoft Edge on my machine. So you can try it out too.

1/2. Enable the WebMCP for testing flag in Edge
3. Go to https://github.com/MicrosoftEdge/webmcp-labs to install the WebMCP explorer extension I used in the demo
4. Test with the pizza demo site, or any other site that defines WebMCP tools.



The question isn't whether AI agents will use your website.
They already do.
The question is whether you, as developers, will help them do it correctly.

We believe WebMCP shifts the conversation to the boundary between agents and real-world systems,
where most failures happen today.

Agentic browsing should be all about humans, agents, and sites working together,
In a trustworthy collaborative way.
And this can only happen if the site developers are part of it.

Let’s build the future of agentic web browsing

- Try the feature
- Join the community group
- Share feedback



We're really excited about it,
And we hope you can join us in building the future of agentic web browsing

So, come join us on the community group and help shape the future of agentic browsing on the web.
- Join the CG: https://www.w3.org/community/webmachinelearning/
- Share feedback on the spec: https://github.com/webmachinelearning/webmcp
- Try the feature.